Linked List

A linked list is a linear data structure where elements, called nodes, are not stored in contiguous memory locations. In a linked list, the linear order is maintained using links or pointers. Each node in a linked list contains at least two parts: a data part and an address part (a pointer) that stores the address of the next node.

Core Components

Node: The basic building block that consists of data and at least one address link.

Head: A special pointer to the first node that marks the beginning of the list. Simply, we can say identifies the first node at the starting of list. If the list is empty, the head is null.

Data: Stores the actual value (integer, string, etc.) to be stored and processed.

Next / Link / Pointer: Stores the address or reference of the next node in the list.

Tail: The last node, which typically points to null to signify the end of the list and does not point to any further node.

Basic Operations on Linked List

A linked list supports several fundamental operations that allow efficient data manipulation. These operations are:

1. Creation

This operation involves creating a linked list by allocating memory for nodes and linking them together. Each node contains data and a pointer to the next node.

2. Traversal

Traversal means visiting each node of the linked list starting from the head node and moving sequentially until the last node (NULL) is reached.

3. Insertion

Insertion is the process of adding a new node to the linked list. It can be performed at:

- Beginning
- End
- Any specific position

4. Deletion

Deletion involves removing a node from the linked list. It can be done:

- From the beginning
- From the end
- From a specific position

5. Display

This operation is used to print or display all the elements (data) of the linked list by traversing from the head node to the last node.

6. Search

Search operation is used to find a specific element in the linked list by traversing node by node and comparing values.

Linked List as an ADT/ Types of Linked List Operations

There are several operations that can be performed on a linked list. They are:

1. insert( ) : The insert( ) operation is used to add a new node into the linked list. Insertion can be performed in different ways:

- At the beginning
- At the end
- At a specific position

During insertion, a new node is created and its link is adjusted to maintain the connection between nodes.

2. delete( ) : The delete( ) operation is used to remove a node from the linked list. Deletion can be performed:

- From the beginning
- From the end
- From a specific position

While deleting, the links are updated, and the removed node is freed from memory.

3. traverse( ) / display( ) : The traverse( ) or display( ) operation is used to visit and print all elements of the linked list from the head to the last node.

4. search( ) : The search( ) operation is used to find whether a particular element exists in the linked list or not. It scans each node sequentially until the element is found or the list ends.

5. isEmpty( ) : The isEmpty( ) operation checks whether the linked list is empty or not.

If the head is equal to NULL, the linked list is empty and the operation returns true. Otherwise, it returns false, indicating that the list is not empty

6. length( ) / count( ) : The length( ) operation counts the number of nodes present in the linked list by traversing the entire list.

Types of linked lists

There are three types of lists in data structures:

1. Singly linked list

2. Doubly linked list

3. Circular linked list

Singly Linked List (SLL)

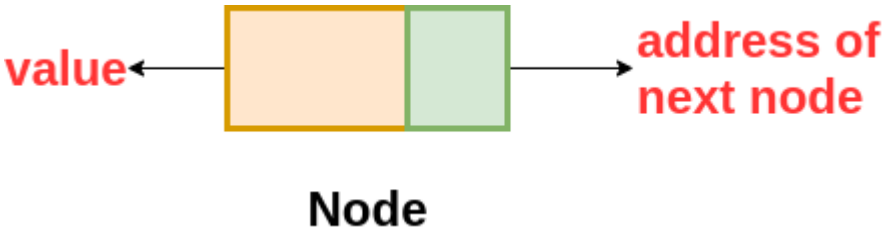
A singly linked list is a linear data structure in which each element, called a node, contains two parts: a data part and an address (link) part. The address part stores the reference to the next node in the list.

In this linked list, each node points only to the next node in a single direction. The first node is accessed using a pointer called the head, and the last node points to NULL, indicating the end of the list.

In this linked list, traversal is possible only in one direction, i.e., from the head node to the last node (NULL), which is called one-way traversal.

Thus, the nodes are connected sequentially, forming a chain-like structure.

In this type of linked list for implementation, we maintain head pointer rather than tail pointer.



<u>Structure of a Node in Singly Linked List (SLL)</u>

We define a node as follows:

```
struct Node {
    int data;
    struct Node *next;
};
```

```
struct Node *head = NULL;
```

Node Insertion Operation in Singly Linked List (SLL)

1. Insert at Beginning/ Front.
2. Insert at End.
3. Insert after a Particular Position
4. Insert at k-th Position/ at a Specified Position.

<u>Algorithm to insert a node at the beginning of a SLL</u>

```
void insertAtBeginning() {
    struct Node *newNode;
    newNode = (struct Node *) malloc(sizeof(struct Node));
    printf("Enter data to insert: ");
    scanf("%d", &newNode->data);
    newNode->next = NULL;s
    newNode->next = head;
    head = newNode;
}
```

Algorithm to insert a node at the end of a SLL

```
void insertAtEnd() {
    struct Node *newNode, *temp;
    newNode = (struct Node *) malloc(sizeof(struct Node));
    printf("Enter data to insert: ");
    scanf("%d", &newNode->data);
    newNode->next = NULL;
    if (head == NULL) {
        head = newNode;
    }
    else {
        temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}
```

Algorithm to insert a node at a particular position of a SLL

```
struct Node {
   int data;
   struct Node *next;
};
struct Node *head = NULL, *newNode, *temp;
void insertAtPosition(int count) {
   int pos, i = 1;
   printf("Enter the position: ");
   scanf("%d", &pos);
   if (pos > count) {
      printf("Invalid Position\n");
   } else {
      temp = head;
      while (i < pos) {
         temp = temp->next;
         i++;
      }
      newNode = (struct Node *) malloc(sizeof(struct Node));
      printf("Enter data: ");
      scanf("%d", &newNode->data);
      newNode->next = temp->next;
      temp->next = newNode;
   }
}
```

Algorithm to insert a node at a k-th position/ at a specified position of a SLL

```
void insertAtPosition() {
   struct Node *newNode, *temp;
   int pos, length = getlength();
   printf("Enter position: ");
   scanf("%d", &pos);
   if (pos < 1 || pos > length + 1) {
      printf("Invalid Position\n");
      return;
   }
   newNode = (struct Node*) malloc(sizeof(struct Node));
   if (newNode == NULL) {
      printf("Memory allocation failed\n");
      return;
   }
   printf("Enter data to insert: ");
   scanf("%d", &newNode->data);
   newNode->next = NULL;
   if (pos == 1) {
      newNode->next = head;
      head = newNode;
   }
   else {
      temp = head;
      for (int i = 1; i < pos - 1; i++) {
         temp = temp->next;
      }
      newNode->next = temp->next;
      temp->next = newNode;
```

```
    }
}
```

Node Deletion Operation in Singly Linked List (SLL)

5. Delete from Beginning.
6. Delete from End.
7. Delete from a Particular Position/ After a Given Location.

<u>Algorithm to delete a node from the beginning of a SLL</u>

```
void deleteFromBeg() {
    struct Node *temp;
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    temp = head;
    head = head->next;
    free(temp);
    printf("Node deleted from the beginning successfully.\n");
}
```

<u>Algorithm to delete a node from the end of a SLL</u>

```
void deleteFromEnd() {
    struct Node *temp, *prevNode;
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    else if (head->next == NULL) {
        temp = head;
        head = NULL;
        free(temp);
        printf("Single node removed successfully.\n");
```

```
    }
    else {
        temp = head;
        while (temp->next != NULL) {
            prevNode = temp;
            temp = temp->next;
        }
        prevNode->next = NULL;
        free(temp);
        printf("Node deleted from end.\n");
    }
}
```

<u>Algorithm to delete a node from a specified position of a SLL</u>

```
void deleteFromPosition() {
    struct Node *temp, *prevNode;
    int pos, i = 1, length = getlength();
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    printf("Enter position: ");
    scanf("%d", &pos);
    if (pos < 1 || pos > length) {
        printf("Invalid Position\n");
        return;
    }
    if (pos == 1) {
        temp = head;
        head = head->next;
        free(temp);
        printf("Node at position 1 deleted.\n");
    }
    else {
```

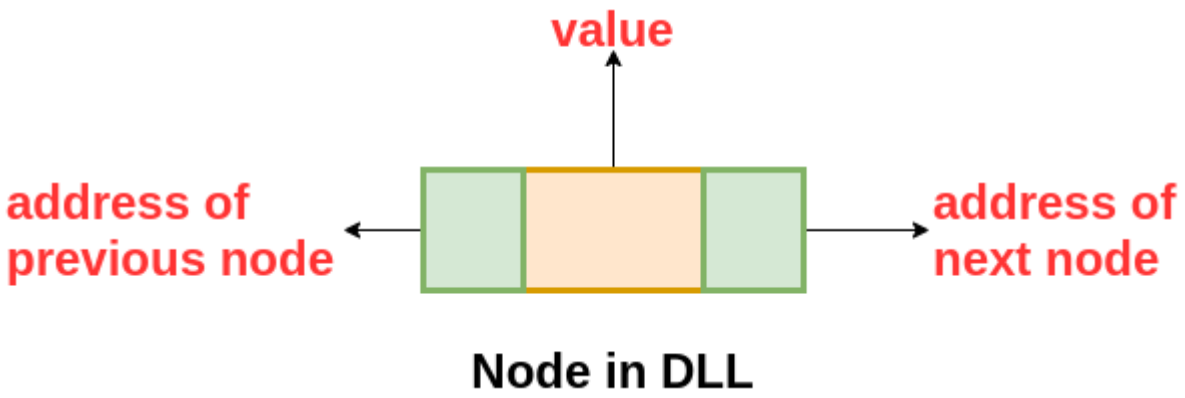
```
        prevNode = head;
        for (i = 1; i < pos - 1; i++) {
            prevNode = prevNode->next;

        }
        temp = prevNode->next;
        prevNode->next = temp->next;
        free(temp);
        printf("Node at position %d deleted.\n", pos);
    }
}
```

Doubly Linked List (DLL)

A doubly linked list is a linear data structure in which each element, called a node, contains three parts: a data part and two address parts. The previous pointer points to the previous node, and the next pointer points to the next node in the list, respectively. In this linked list, each node points to both the next node and the previous node, allowing traversal in both directions. The first node is accessed using a pointer called the head, and the last node's next pointer is set to NULL, indicating the end of the list and this last node is accessed using a pointer called tail. Similarly, the prev pointer of the first node is NULL, indicating the start of the list.

In this linked list, traversal is possible in both directions, i.e., from the head node to the last node and from the last node back to the head, which is called two-way traversal.



One drawback is that each node requires extra memory for the previous pointer, which stores the address of the previous node. In this type of linked list for implementation, we maintain both head and tail pointers.

Structure of a Node in Doubly Linked List (DLL)

We define a node as follows:

```
struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
};
struct Node *head = NULL;
struct Node *tail = NULL;
```

Node Insertion Operation in Doubly Linked List (DLL)

1. Insert at Beginning/ Front.
2. Insert at End.
3. Insert at k-th Position/ at a Specified Position.

Algorithm to insert a node at the beginning of a DLL

```
void insertAtBeginning() {
    struct Node *newNode;
    newNode = (struct Node *) malloc(sizeof(struct Node));
    printf("Enter data to insert: ");
    scanf("%d", &newNode->data);
```

```
    newNode->prev = NULL;
    newNode->next = NULL;
    if (head == NULL) {
        head = tail = newNode;
    }
    else {
        newNode->next = head;
        head->prev = newNode;
        head = newNode;
    }
}
```

<u>Algorithm to insert a node at the end of a DLL</u>

```
void insertAtEnd() {
    struct Node *newNode;
    newNode = (struct Node *) malloc(sizeof(struct Node));

    printf("Enter data: ");
    scanf("%d", &newNode->data);
    newNode->next = NULL;

    if (head == NULL) {
        newNode->prev = NULL;
        head = tail = newNode;
    }
    else {
        tail->next = newNode;
        newNode->prev = tail;
        tail = newNode;
```

```
    }
}
```

<u>Algorithm to insert a node at a k-th position/ at a specified position of a DLL</u>

```
void insertAtPosition() {
    struct Node *newNode, *temp;
    int pos, length = getlength();
    printf("Enter position: ");
    scanf("%d", &pos);
    if (pos < 1 || pos > length + 1) {
        printf("Invalid Position\n");
        return;
    }
    newNode = (struct Node*) malloc(sizeof(struct Node));
    printf("Enter data: ");
    scanf("%d", &newNode->data);
    if (pos == 1) {
        newNode->next = head;
        newNode->prev = NULL;
        if (head != NULL) {
            head->prev = newNode;
        }
        head = newNode;
    }
    else {
        temp = head;
        for (int i = 1; i < pos - 1; i++) {
            temp = temp->next;
        }
        newNode->next = temp->next;
```

```
        newNode->prev = temp;
        if (temp->next != NULL) {
            temp->next->prev = newNode;
        }
        temp->next = newNode;
    }
}
```

Node Deletion Operation in Doubly Linked List (DLL)

1. Delete from Beginning.
2. Delete from End.
3. Delete from a Particular Position/ After a Given Location.

<u>Algorithm to delete a node from the beginning of a DLL</u>

```
void deleteFromBeg() {
    struct Node *temp;
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    temp = head;
    head = head->next;
    if (head != NULL) {
        head->prev = NULL;
    }
    free(temp);
    printf("Node deleted from the beginning successfully.\n");
}
```

Algorithm to delete a node from the end of a DLL

```
void deleteFromEnd() {
    struct Node *temp, *prevNode;
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    else if (head == tail) {
        head = tail = NULL;
        printf("Single node removed successfully.\n");
    }
    else {
        temp = head;
        while (temp->next != NULL) {
            prevNode = temp;
            temp = temp->next;
        }
        prevNode->next = NULL;
        tail = prevNode;
        free(temp);
        printf("Node deleted from end.\n");
    }
}
```

Algorithm to delete a node from a specified position of a DLL

```
void deleteFromPosition() {
    struct Node *temp, *prevNode;
    int pos, i = 1, length = getlength();
    if (head == NULL) {
```

```
        printf("List is empty\n");
        return;
    }
    printf("Enter position: ");
    scanf("%d", &pos);
    if (pos < 1 || pos > length) {
        printf("Invalid Position\n");
        return;
    }
    if (pos == 1) {
temp = head;
    head = head->next;
    if (head != NULL) {
        head->prev = NULL;
    }
        free(temp);
        printf("Node at position 1 deleted.\n");
    }
    else {
        prevNode = head;
        for (i = 1; i < pos - 1; i++) {
            prevNode = prevNode->next;

        }
        temp = prevNode->next;
        prevNode->next = temp->next;
         if (temp->next != NULL) {
            temp->next->prev = prevNode;
        }
        free(temp);
        printf("Node at position %d deleted.\n", pos);
    }
}
```

Circular linked list (CLL)

Generally, there are two types of circular linked lists, namely **singly circular linked list** and **doubly circular linked list**

A circular linked list is a linear data structure where the last nodes points back to the first node instead of pointing to NULL, forming a continuous loop.

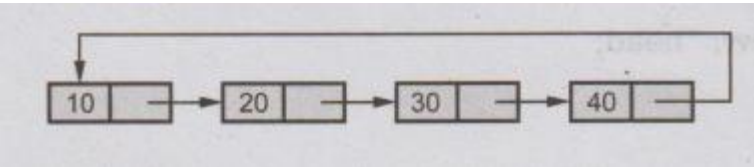
NULL, forming a continuous loop. This structure allows for infinite traversal without ever reaching an "end". Ideal for tasks like scheduling and managing playlists, where smooth and repeated.

Types of Circular Linked Lists

- **Singly Circular Linked List:** Each node contains data and a single "next" pointer. The "next" pointer of the last node is linked to the head node.
- **Circular Doubly Linked List:** Each node contains data and two pointers: "next" and "previous." The last node's "next" points to the head, and the head's "previous" points back to the last node, allowing bidirectional traversal.

Single Circular linked list (SCLL)

A singly circular linked list is a variation of a linked list where the address part of the last node points to the first node instead of pointing to NULL, forming a closed circular loop. In this type of linked list for implementation, we usually maintain a tail pointer rather than just a head pointer.



Structure of a Node in Singly Circular Linked List (SCLL)

We define a node as follows:

```
struct Node {
    int data;
    struct Node *next;
};
struct Node *head = NULL;
```

Node Insertion Operation in Singly Circular Linked List (SCLL)

1. Insert at Beginning/ Front.
2. Insert at End.
3. Insert at k-th Position/ at a Specified Position.

Algorithm to insert a node at the beginning of a SCLL

```
void insertAtBeginning() {
    struct Node *newNode;
    newNode = (struct Node *) malloc(sizeof(struct Node));
    printf("Enter data to insert: ");
    scanf("%d", &newNode->data);
    if (tail == NULL) {
        tail = newNode;
        tail->next = newNode;
    }
    else {
        newNode->next = tail->next;
        tail->next = newNode;
    }
}
```

Algorithm to insert a node at the end of a SCLL

```
void insertAtEnd() {
    struct Node *newNode;
    newNode = (struct Node *) malloc(sizeof(struct Node));
    printf("Enter data to insert: ");
    scanf("%d", &newNode->data);
    if (tail == NULL) {
        tail = newNode;
        tail->next = newNode;
    }
    else {
        newNode->next = tail->next;
        tail->next = newNode;
        tail = newNode;
    }
}
```

Algorithm to insert a node at a k-th position/ at a specified position of a SCLL

```
void insertAtPosition() {
    struct Node *newNode, *temp;
    int pos, i, length = getlength();
    printf("Enter position: ");
    scanf("%d", &pos);
    if (pos < 1 || pos > length + 1) {
        printf("Invalid Position\n");
        return;
    }
```

```
    newNode = (struct Node*) malloc(sizeof(struct Node));
    printf("Enter data to insert: ");
    scanf("%d", &newNode->data);
    if (tail == NULL) {
        tail = newNode;
        tail->next = newNode;
    }
    else if (pos == 1) {
        newNode->next = tail->next;
        tail->next = newNode;
    }
    else {
        temp = tail->next;
        for (i = 1; i < pos - 1; i++) {
            temp = temp->next;
        }
        newNode->next = temp->next;
        temp->next = newNode;
        if (temp == tail) {
            tail = newNode;
        }
    }
}
```

Node Deletion Operation in Singly Circular Linked List (SCLL)

1. Delete from Beginning.
2. Delete from End.
3. Delete from a Particular Position/ from a Given Location.

<u>Algorithm to delete a node from the beginning of a SCLL</u>

```
void deleteFromBeg() {
    struct Node *temp;
    if (tail == NULL) {
        printf("List is empty.\n");
        return;
    }
    temp = tail->next;
    if (tail->next == tail) {
        tail = NULL;
        free(temp);
    }
    else {
        tail->next = temp->next;
        free(temp);
    }
    printf("Node deleted from the beginning successfully.\n");
}
```

Note:

tail->next == tail ⇔ temp == tail

Algorithm to delete a node from the end of a SCLL

```
void deleteFromEnd() {
    struct Node *temp, *prevNode;
    if (tail == NULL) {
        printf("List is empty.\n");
        return;
    }
    temp = tail->next;
    if (temp == tail) {
        tail = NULL;
        free(temp);
        printf("Single node removed successfully.\n");
    }
    else {
        while (temp->next != tail->next) {
            prevNode = temp;
            temp = temp->next;
        }
        prevNode->next = tail->next;
        tail = prevNode;
        free(temp);
        printf("Node deleted from end.\n");
    }
}
```

Algorithm to delete a node from a specified position of a SCLL

```
void deleteFromPosition() {
    struct Node *temp, *prevNode;
    int pos, i = 1, length = getlength();
    if (tail == NULL) {
        printf("List is empty\n");
```

```
    return;
}
printf("Enter position: ");
scanf("%d", &pos);
if (pos < 1 || pos > length) {
    printf("Invalid Position\n");
    return;
}

if (pos == 1) {
    temp = tail->next;
    if (temp == tail) {
        tail = NULL;
    }
    else {
        tail->next = temp->next;
    }
    free(temp);
    printf("Node at position 1 deleted.\n");
    return;
}
prevNode = tail->next;
for (i = 1; i < pos - 1; i++) {
    prevNode = prevNode->next;
}
temp = prevNode->next;
prevNode->next = temp->next;

if (temp == tail) {
    tail = prevNode;
}

free(temp);
printf("Node at position %d deleted.\n", pos);
```

}

Pointer Update Steps for Insertion and Deletion

<u>For insertion:</u>

1. First, update the address part of the independent node (new node). If there are two pointers (prev and next), update the next pointer first and then the prev pointer, since the new node is not yet part of the list and it is safe.
2. Then, update the address part of the dependent (existing) nodes that are already in the list.

   After successfully updating the address part of the new node, when the turn goes to the existing nodes and it is not clear whether to update the leftmost or rightmost node first, always update the rightmost node link first to avoid breaking the chain.
3. Finally, update the structure pointer (head or tail) at the end.

<u>For deletion:</u>

In deletion, the target node is already part of the list, so removing it directly may break links. Therefore, before deleting the target node, always check the address part of its neighbouring nodes and fix the links first. Then safely remove the node. If the tail pointer is pointing to the last node, then after deleting that last node, the tail must be moved to the previous node so that it correctly points to the new last node.

What is the difference between Linked List and Linear Array?

S. No.	Array	Linked List
1.	Insertions and deletions are difficult.	Insertions and deletions can be done easily.
2.	It needs movements of elements for insertion and deletion	It does not need movement of nodes for insertion and deletion.
3.	In it space is wasted	In it space is not wasted.
4.	It is more expensive	It is less expensive.
5.	It requires less space as only information is stored	It requires more space as pointers are also stored along with information.
6.	Its size is fixed	Its size is not fixed.
7.	It cannot be extended or reduced according to requirements	It can be extended or reduced according to requirements.
8.	Same amount of time is required to access each element	Different amount of time is required to access each element.
9.	Elements are stored in consecutive memory locations.	Elements may or may not be stored in consecutive memory locations.
10.	If have to go to a particular element then we can reach their directly.	If we have to go to a particular node then we have to go through all those nodes that come before that node.

Advantages of Linked Lists

Below are some of the advantages of linked lists:

- Dynamic size
- Efficient insertions and deletions
- No memory waste
- Flexibility in memory allocation
- Easy implementation of abstract data types
- Flexibility in rearrangement

Disadvantages of Linked Lists

Below are some of the disadvantages of linked lists:

- Random access is inefficient
- Extra memory usage
- Reverse traversal is difficult
- Cache performance
- No binary search
- Vulnerable to memory leaks

Applications of Linked List

Linked lists are used in many areas where dynamic memory and flexible data structures are needed.

- Dynamic memory allocation
- Implementation of stack and queue
- Polynomial representation
- Sparse matrix representation
- Undo/redo operations
- Navigation (forward/backward traversal)
- Graph representation (adjacency list)
- File system (linked file allocation)